

4.2 — Apply Few-Shot Prompting to Improve Output Consistency

Concept

Domain mental model: Specificity beats vagueness. Examples beat instructions. Schemas beat parsing.

Few-shot prompting is the practice of embedding concrete input→output examples directly in the prompt. Rather than describing what you want in abstract terms, you demonstrate it. The model generalises from the examples to handle new inputs in the same style, format, and register.

This technique is the single most reliable lever for output consistency. Instructions alone are ambiguous — the phrase "return a concise summary" means different things to different readers. An example that shows exactly how long, what voice, and what structure is unambiguous.

Why Examples Beat Instructions

Instructions describe a rule. Examples instantiate it. When format consistency matters — structured JSON, a specific markdown layout, a predictable error shape — an instruction like "return JSON with a `status` and `message` field" still leaves room for variation in casing, nesting, extra fields, and null handling. A single example eliminates most of that variance. Two or three examples eliminate almost all of it.

The model learns locally: it pattern-matches the examples in context window, not a memorised rulebook. This means you can override general tendencies with a few well-chosen examples, which is exactly what you want when your format deviates from common conventions.

Targeting Ambiguous Cases

Generic examples teach the happy path. **Targeted examples** teach the edge cases you actually care about. If your tool call input can be either a string or an array, include one example of each. If an optional field can be omitted or null, show both. If a classification label has a boundary case that is easy to mis-categorise, include an example right at that boundary.

The principle: identify the ambiguous or error-prone input space, then write examples that cover it explicitly. A single well-targeted example for a tricky case is worth more than three examples of the easy case.

How Many Examples

- **1 example:** Establishes format, useful for simple schemas.
- **2–3 examples:** The practical default. Covers the happy path plus at least one variation.
- **4–5 examples:** Appropriate when the output space is complex, the edge cases are numerous, or the task is highly sensitive to format consistency.

- **More than 5:** Rarely necessary and increases prompt token cost. Prefer a concise schema plus 2–3 targeted examples over 8 generic ones.

Adding more examples beyond the point of diminishing returns does not improve consistency — it increases context and cost without benefit. Stop adding examples when the model already produces correct output on your test inputs.

Placement in the System Prompt

Place few-shot examples at the **end of the system prompt**, immediately before the first human turn. This positions them closest to the model's first output, maximising their influence on format. Examples buried in the middle of a long system prompt are partially overridden by later instructions.

If the examples are long, wrap them in clear delimiters (XML tags are reliable) so the model can distinguish examples from directives.

Code Example

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

// Target: classify customer feedback as POSITIVE, NEGATIVE, or NEUTRAL
// with a confidence score (0.0–1.0) and a one-sentence rationale.
// Edge case: sarcastic positive phrasing should resolve to NEGATIVE.

const systemPrompt = `You classify customer feedback. Return JSON only.

<examples>
Input: "Delivery was two days early and the packaging was perfect."
Output: {"label": "POSITIVE", "confidence": 0.97, "rationale": "Explicit praise for delivery speed and packaging quality."}

Input: "It sort of works I guess."
Output: {"label": "NEUTRAL", "confidence": 0.72, "rationale": "Faint, non-committal language suggesting mixed feelings."}

Input: "Oh great, another item that broke on day one. Really excellent quality."
Output: {"label": "NEGATIVE", "confidence": 0.91, "rationale": "Sarcastic positive phrasing paired with a complaint about a broken item."}
</examples>`;

async function classifyFeedback(feedback: string) {
  const response = await client.messages.create({
    model: "claude-sonnet-4-5",
    max_tokens: 256,
    system: systemPrompt,
    messages: [
      {
        role: "user",
```

```

        content: feedback,
      },
    ],
  });

  const raw = response.content[0];
  if (raw.type !== "text") {
    throw new Error(`Unexpected content type: ${raw.type}`);
  }

  return JSON.parse(raw.text) as {
    label: "POSITIVE" | "NEGATIVE" | "NEUTRAL";
    confidence: number;
    rationale: string;
  };
}

```

Three examples cover the happy path (clear positive), the ambiguous boundary (neutral hedging), and the targeted edge case (sarcasm). The model now handles all three reliably without additional instruction. The output schema is never described in prose — the examples carry that information entirely.

Common Mistakes

Using only happy-path examples. If all your examples show clean, unambiguous inputs, the model has no guidance for messy real-world data. Identify the cases that are most likely to go wrong and write examples for those specifically.

Describing the format in prose and also showing it in examples. Redundancy between instructions and examples creates a potential contradiction if they ever diverge. Prefer to let the examples carry the format definition. Add prose instructions only for things that cannot be shown (e.g. "never include PII in the rationale field").

Placing examples too early in the system prompt. Examples placed in the first third of a long prompt are partially overridden by the instructions that follow them. Move them to the bottom, just before the first human turn, to maximise recency.

Using inconsistent example structure. If one example uses `"label"` and another uses `"classification"`, you have taught the model nothing stable. Every example must follow the exact same schema with the exact same field names and value formats.

Adding more examples when the issue is a bad schema. If output is inconsistent despite several examples, the problem is usually that the output format itself is ambiguous or under-constrained. Define a strict schema (e.g. use a JSON Schema or a TypeScript type as a reference) rather than adding more examples to compensate.

Relying on few-shot alone for high-stakes structured output. For critical integrations, combine few-shot examples with Claude's native tool-use / structured output feature. Few-shot improves consistency; a machine-validated schema enforces it.

References

- Anthropic prompt engineering guide: Few-shot prompting
- Anthropic documentation: System prompts and message structure
- Exam domain 4: Prompt Engineering Techniques
- See also: 4.1 — Structure System Prompts for Clarity and Role Separation